Technical Design Description

Technical Design Specifications for Haat_Bazar Architecture

1. Introduction

1.1 Purpose

This Technical DesignDescription (tdd) details the technical architecture and object-oriented design for Haat_Bazar, an agricultural digital marketplace. It serves to guide developers in implementing the backend services, dynamic pricing algorithms, and mobile clients outlined in the project requirements.

1.2 Scope

This document covers the design of the Haat_Bazar MVP. It includes the decomposition of the system into objects and services, data persistence layers (PostgreSQL), and the interface definitions between the client applications and the backend API gateway.

1.3 References

- Haat_Bazar System Architecture Specifications (Oct 2026)
- Haat_Bazar Operational Mathematical & Pricing Models (Oct 2026)
- IEEE Std 1016-1987, Recommended Practice for Technical DesignDescriptions
- IEEE Std 1016.1-1993, Guide to Technical DesignDescriptions

2. Design Views Organization

Following the Layered Virtual Machine/Object-Oriented Design (LVM/OOD) method, the design is structured into four primary views to satisfy the information needs of the development team:

1. **Decomposition Description:** Identifies the primary system components and objects.
2. **Dependency Description:** Illustrates how objects interact and rely on system resources.
3. **Interface Description:** Defines the API contracts.
4. **Detail Description:** Provides the internal data structures and processing algorithms.

3. Decomposition Description

The system is decomposed into the following core design entities (objects/services):

1. **User Identity Manager:** Handles registration, authentication, and role assignment (Farmer, Consumer, Wholesaler, Hub Operator).
2. **Marketplace Engine (Product Listing):** Manages the lifecycle of harvest batches, including creation, availability tracking, and search indexing.
3. **MOQ & Cart Guard:** Validates shopping carts against dynamic Minimum Order Quantities based on buyer type.
4. **Pricing & Logistics Calculator:** Computes the final transaction cost (Base + Service + Delivery).
5. **Order & Hub Fulfillment Manager:** Orchestrates order state transitions, escrow locks, and physical hub verification events.

4. Dependency Description

Component	Upstream Dependencies	Primary Responsibility
Marketplace Engine	User Identity Manager	Verifies seller permissions before batch registration

Component	Upstream Dependencies	Primary Responsibility
MOQ & Cart Guard	User Identity Manager	Enforces consumer (5kg) vs. wholesale (50kg) constraints
Pricing & Logistics Calculator	Marketplace Engine, Go...	Computes dynamic crop base prices and geospatial routing costs
Order & Hub Fulfillment Manager	bKash/Nagad MFS Gate...	Confirms escrow locks before signaling hub dispatch

5. Interface Description

The system exposes RESTful HTTPS endpoints via an API Gateway.

`POST /api/v1/harvests/`

Used by the Farmer App to create a new product listing. Requires multipart/form-data for image uploads.

GET /api/v1/marketplace/

Used by Buyer Apps to retrieve available inventory. Supports geospatial querying (PostGIS) to return local results based on distance thresholds.

`POST /api/v1/cart/validate/`

Used by the Buyer App during checkout.

- **Input:** Cart items array, Buyer ID.
- **Output:** `200 OK` (if MOQ met) OR `400 Bad Request` with a deficit weight prompt.

`POST /api/v1/orders/checkout/`

Initiates the MFS payment flow and transitions inventory state to 'Reserved'.

6. Detail Description

6.1 Data Structure (PostgreSQL schema extract)

```
CREATE TABLE product_listings (

    id SERIAL PRIMARY KEY,

    farmer_id INT REFERENCES users(id),

    product_name VARCHAR(100) NOT NULL,

    total_batch_quantity DECIMAL(10,2) NOT NULL,

    remaining_quantity DECIMAL(10,2) NOT NULL,

    farmer_base_price_per_kg DECIMAL(10,2) NOT NULL,

    consumer_moq_kg DECIMAL(5,2) DEFAULT 5.00,

    wholesaler_moq_kg DECIMAL(10,2) DEFAULT 50.00

);
```

6.2 Processing: Pricing & Logistics Calculator Algorithm

When an order total is requested, the Pricing & Logistics Calculator performs the following execution pipeline:

1. **Calculate Base Cost:**
   Sum the total crop cost across all cart line items:
   $$\text{Base Cost} = \sum (\text{ordered_quantity}_i \times \text{farmer_base_price_per_kg}_i)$$

2. **Calculate Service Fee:**
   Multiply the computed Base Cost by the active platform commission tier (e.g., 0.05 for $5%$).
3. **Calculate Delivery Fee:**
   Determine the linear/route distance from the assigned `hub_id` to the buyer's `gps_location` using PostGIS spatial routing. Apply the dynamic cost-per-kilometer rate structure.
4. **Finalize Transaction Total:**
   Aggregate all three components ($\text{Base} + \text{Service} + \text{Delivery}$) and transmit the structured payload to the client interface for MFS authorization.